WordPress Performance Optimization Checklist

Version 1.0 — DRAFT

Dan Knauss, General Editor

WordPress Performance Optimization Checklist

> Status: DRAFT
> Version: 1.0
> General Editor: Dan Knauss

A practical, operations-oriented guide for improving WordPress performance without guessing. Work through this in order: measure first, reduce unnecessary work, cache safely, optimize delivery, then verify with the same measurements.

> Principle: performance is not a score. Optimize for the user experience: fast response, quick interactivity, stable layout, and predictable behavior across real devices and networks.

Table of Contents

Companion cross-reference: WP VIP Enterprise Performance

Use this checklist as the broad WordPress/operator workflow. When the target site is enterprise-scale, VIP-hosted, or dominated by backend/database/cache behavior, cross-check the companion WP VIP runbook at `wpvip-enterprise-performance-operational-checklist.md`. It goes deeper on `WP_Query`, `NOT IN`, `LIKE`, post meta, taxonomies, `EXPLAIN`, Elasticsearch/offloaded search, object-cache race conditions, cache stampedes, load testing, New Relic/APM, and high-traffic events.

---

0. Safety and scope

Before changing anything, define the target and guardrails.

- [ ] Identify the environment: local, staging, production, multisite, WooCommerce, membership, LMS, or other dynamic site type.
- [ ] Identify the slow experience:

  - [ ] public page
  - [ ] logged-in page
  - [ ] WordPress admin screen
  - [ ] WooCommerce cart/checkout/account
  - [ ] REST API endpoint
  - [ ] `admin-ajax.php` request
  - [ ] WP-Cron or background job

- [ ] Pick 1–3 exact URLs or routes to test repeatedly.
- [ ] Decide what changes are allowed:

  - [ ] read-only diagnostics
  - [ ] configuration changes
  - [ ] plugin/theme changes
  - [ ] database cleanup
  - [ ] hosting/CDN changes

- [ ] Confirm rollback options:

  - [ ] database backup
  - [ ] file backup or git branch
  - [ ] ability to disable new rules/plugins
  - [ ] cache/CDN rule rollback

---

1. Baseline measurement

Capture current behavior before optimizing.

User-facing metrics

- [ ] Measure Core Web Vitals for important templates:

  - [ ] LCP — Largest Contentful Paint
  - [ ] INP — Interaction to Next Paint
  - [ ] CLS — Cumulative Layout Shift

- [ ] Compare lab and field data:

  - [ ] Lighthouse/PageSpeed Insights for controlled testing
  - [ ] CrUX or real user monitoring for real-user data where available

- [ ] Record device/network assumptions: desktop, mobile, throttled mobile, logged-in, logged-out.

Server/backend metrics

- [ ] Measure uncached and cached TTFB separately.
- [ ] Run the same request multiple times to separate cold-cache from warm-cache behavior.
- [ ] Check response headers:
  - [ ] cache status: `HIT`, `MISS`, `BYPASS`, `DYNAMIC`, etc.
  - [ ] `Cache-Control`
  - [ ] cookies
  - [ ] redirects
  - [ ] CDN headers
- [ ] If WP-CLI and the relevant command packages are available, collect profiling data:

Availability check. `wp profile` and `wp doctor` are not bundled with every WP-CLI install; they ship as separate command packages. Check availability before using them:

```
wp cli has-command profile && echo "profile available"
wp cli has-command doctor && echo "doctor available"
# Install if missing and your environment allows package installs:
# wp package install wp-cli/profile-command
# wp package install wp-cli/doctor-command
```

If neither command nor package install is available (common on managed hosts), fall back to Query Monitor, APM traces, host-provided profilers, or in-code `microtime()` timers.

```
wp profile stage --url="https://example.com/target-page/"
wp profile hook --url="https://example.com/target-page/"
wp doctor check
```

- [ ] If Query Monitor is available, inspect:
  - [ ] total query count
  - [ ] slow queries
  - [ ] duplicate queries
  - [ ] HTTP API calls
  - [ ] hooks/components responsible for time
  - [ ] object cache statistics

Record baseline values in two small tables. Page-render routes can have Core Web Vitals; API and server routes generally do not.

Page-render baseline:

Target	State	TTFB	LCP	INP	CLS	Cache status	Notes
`/`	logged out, warm cache						
`/shop/`	logged out, warm cache						
Other key page routes							

Server / API route baseline:

Target	State	TTFB	Payload size	Cache status	Notes
`/wp-json/...`	logged in/out				
`/wp-admin/admi ajax.php?a`	varies				
Other dynamic endpoints					

---

2. Request path and DNS

A page cannot be fast if connection setup is slow or unreliable.

- [ ] Use a reputable DNS provider with fast global resolution.
- [ ] Check DNS records for unnecessary indirection.

- [ ] Keep the DNS path simple:
  - [ ] avoid long CNAME chains
  - [ ] remove stale records
  - [ ] ensure apex/root and `www` behavior is intentional
- [ ] Set TTLs intentionally:
  - [ ] lower TTL before planned migrations
  - [ ] higher TTL for stable records
- [ ] Confirm IPv4/IPv6 behavior if both are configured.
- [ ] Use DNS prefetch only for origins the page will actually use.
- [ ] Use preconnect sparingly for critical third-party origins that are needed early.

Examples:

```
<link rel="dns-prefetch" href="//fonts.gstatic.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
```

Checklist:

- [ ] Remove preconnects for non-critical third parties.
- [ ] Do not preconnect to many domains; each connection has a cost.
- [ ] Confirm third-party origins are still needed before optimizing around them.

---

3. TLS, HTTP, and redirects

Transport setup affects TTFB and resource delivery.

- [ ] Enforce HTTPS consistently.
- [ ] Remove redirect chains:
  - [ ] `http://` -> `https://`
  - [ ] apex -> `www` or `www` -> apex
  - [ ] trailing slash canonicalization
- [ ] Confirm the final canonical URL is reached in one redirect or fewer.
- [ ] Ensure HTTP/2 or HTTP/3 support is enabled where appropriate.
- [ ] Keep certificates valid and automatically renewed.
- [ ] Use HSTS only when HTTPS is fully correct across subdomains.
- [ ] Check that CDN and origin TLS settings are compatible.

Useful checks:

```
curl -I -L https://example.com/
curl -w "namelookup:%{time_namelookup} connect:%{time_connect} tls:%{time_appconne
o /dev/null -s https://example.com/
```

---

4. Hosting and server layer

Good hosting reduces the amount of work WordPress has to fight through.

- [ ] Confirm adequate PHP workers for traffic and dynamic pages.
- [ ] Use a supported PHP version and modern database version.
- [ ] Enable PHP OPcache.
- [ ] Verify memory limits are sufficient but not hiding runaway code.
- [ ] Check error logs for repeated warnings, fatals, retries, or timeouts.
- [ ] Confirm server-level page cache or CDN cache integration.
- [ ] Check whether object cache is available and persistent.
- [ ] Confirm backups, security scans, and maintenance tasks are not running during peak traffic.

OPcache checks to discuss with hosting/ops:

- [ ] OPcache enabled for web requests.
- [ ] OPcache memory not exhausted.
- [ ] OPcache revalidation settings appropriate for the deployment workflow.

---

5. Full-page caching / HTML caching

Cache-layer terminology

Three distinct WordPress caching subsystems are often conflated. `WP_CACHE` is a constant that allows WordPress to load the `wp-content/advanced-cache.php` drop-in for page caching (saving complete rendered HTML). `wp-content/object-cache.php` is a separate drop-in for persistent object caching (Redis/Memcached, caching database/query/application objects across requests). The Transients API uses object cache when persistent caching is configured, otherwise falls back to the `wp_options` table. See `REFERENCE-WP-Transients-Persistent-Object-Cache.md` and `DEVELOPER_REFERENCE.md` §4 for full treatment.

Full-page caching is powerful because it can skip WordPress execution entirely for cacheable requests.

- [ ] Identify which pages can be cached as full HTML:
  - [ ] homepage
  - [ ] posts/pages
  - [ ] category/tag archives
  - [ ] product listing pages when not personalized
  - [ ] landing pages
- [ ] Identify which pages should bypass full-page cache:
  - [ ] `/wp-admin/`
  - [ ] cart
  - [ ] checkout
  - [ ] account pages
  - [ ] pages with user-specific private content
  - [ ] preview URLs
- [ ] Confirm cache headers are intentional.
- [ ] Confirm logged-out pages get `HIT` after warming.
- [ ] Confirm logged-in or personalized pages do not leak private HTML.
- [ ] Warm important pages after deploys or cache purges.
- [ ] Avoid global cache purges unless necessary.

Query-string cache fragmentation

Ignore or normalize marketing query strings when they do not change page content.

Examples that commonly fragment cache keys:

```
/shop?utm_source=newsletter&utm_medium=email&utm_campaign=spring_sale
/shop?fbclid=IwAR1XyZExampleValue
/shop?gclid=EAIaIQobChMIExample
/shop?utm_source=facebook&utm_content=carousel_ad_3
/shop?lang=en
/shop?preview=true
/shop?utm_source=newsletter&utm_medium=email&fbclid=IwAR1XyZExampleValue
```

Checklist:

- [ ] Ignore `utm_*` parameters for HTML cache when content does not vary.
- [ ] Ignore ad click IDs such as `fbclid` and `gclid` when safe.
- [ ] Never cache preview URLs as public canonical HTML.
- [ ] Treat language, currency, region, and personalization parameters carefully; they may change content.

Cookie cache fragmentation

- [ ] List cookies set on public pages.
- [ ] Remove cookies that are not required before interaction.
- [ ] Configure cache bypass only for cookies that truly indicate personalization/session state.
- [ ] Audit plugins that set cookies globally.
- [ ] Avoid letting analytics or marketing cookies bypass HTML cache.

---

6. CDN and edge delivery

A CDN helps when it caches the right assets and avoids unnecessary trips to origin.

- [ ] Put static assets behind the CDN:
  - [ ] images
  - [ ] CSS
  - [ ] JavaScript
  - [ ] fonts
- [ ] Confirm immutable/static assets have long cache lifetimes.
- [ ] Use file versioning or hashed filenames for assets that change.
- [ ] Configure HTML cache rules intentionally; do not rely on defaults blindly.
- [ ] Track cache hit ratio.
- [ ] Investigate low hit ratio:
  - [ ] too many query strings
  - [ ] unnecessary cookies
  - [ ] short TTLs
  - [ ] frequent purges
  - [ ] personalized HTML
  - [ ] inconsistent cache keys
- [ ] Confirm origin shielding or tiered caching if traffic is global/high-volume.
- [ ] Verify CDN does not break admin, checkout, REST, or logged-in flows.

---

7. WordPress runtime

Optimize what WordPress loads and executes for each request.

- [ ] Profile request stages: bootstrap, main query, template, hooks.
- [ ] Identify slow hooks/callbacks.
- [ ] Remove unnecessary plugins or modules.
- [ ] Disable plugin features not used on the site.
- [ ] Prevent frontend loading of admin-only functionality.
- [ ] Avoid running expensive logic on every request.
- [ ] Move heavy work to background jobs where appropriate.
- [ ] Cache expensive computed results.
- [ ] Avoid remote HTTP calls during page generation unless cached and timeout-protected.

WP-CLI profiling examples:

```
wp profile stage --url="https://example.com/sample-page/"
wp profile hook --url="https://example.com/sample-page/" --all
```

Code review checklist:

- [ ] Are hooks scoped to the pages where they are needed?
- [ ] Are queries run inside loops?
- [ ] Are external APIs called during render?
- [ ] Are transients/object cache used for expensive repeat work?
- [ ] Are cache keys specific enough but not over-fragmented?
- [ ] Are large options autoloaded unnecessarily?

---

8. Database and query performance

Database optimization is about reducing expensive work, not just “cleaning tables.”

- [ ] Check total query count per request.
- [ ] Find slow queries.
- [ ] Find duplicate queries.
- [ ] Identify N+1 patterns.
- [ ] Review `WP_Query`, meta queries, taxonomy queries, and search queries.
- [ ] Avoid broad unbounded queries.
- [ ] Paginate large result sets.
- [ ] Cache expensive query results when safe.
- [ ] Avoid using post meta as a high-volume relational query system.
- [ ] Add indexes only after confirming query patterns and testing impact.

Autoloaded options:

- [ ] Measure total autoloaded option size.
- [ ] Identify the largest autoloaded options.
- [ ] Set large rarely-used options to `autoload = off` where safe (WordPress 6.6+ vocabulary; older rows may still use `yes`/`no` — see Appendix A).
- [ ] Remove orphaned plugin/theme options only after backup and verification.

Useful SQL inspection examples (updated for WordPress 6.6+):

> Note: As of WordPress 6.6 (June 2024), the `autoload` column can hold five values: `'on'`, `'off'`, `'auto'`, `'auto-on'`, `'auto-off'`. Older rows still use `'yes'`/`'no'`. Filters that only look for `'yes'` miss everything written under the new vocabulary. The queries below cover both. See the Modern additions appendix at the end of this checklist for the full 6.6 / 6.8 update list.

```
SELECT option_name, LENGTH(option_value) AS size, autoload
FROM wp_options
WHERE autoload IN ('yes', 'on', 'auto', 'auto-on')
ORDER BY size DESC
LIMIT 20;
SELECT SUM(LENGTH(option_value)) AS autoload_bytes
FROM wp_options
WHERE autoload IN ('yes', 'on', 'auto', 'auto-on');
```

To change autoload state, use the WP 6.4+ API rather than hand-editing the column:

```
wp_set_option_autoload( 'my_huge_option', false );
wp_set_options_autoload( array( 'opt_a', 'opt_b' ), false );
```

Or via WP-CLI:

```
wp option set-autoload my_huge_option no
```

---

9. Object caching and transients

Object caching reduces repeated database work. Persistent object caching keeps those benefits across requests.

- [ ] Check whether a persistent object cache drop-in exists: `wp-content/object-cache.php`.
- [ ] Confirm Redis or Memcached is healthy if used.

- [ ] Monitor object cache hit/miss ratio.
- [ ] Cache expensive computations and query results.
- [ ] Avoid caching highly personalized data with broad keys.
- [ ] Use stable, explicit cache keys.
- [ ] Set appropriate expirations.
- [ ] Avoid storing huge objects if they cause memory pressure.

Transients checklist:

- [ ] Use transients for expensive results that can expire.
- [ ] Do not autoload large transient-like data unnecessarily.
- [ ] Clean up expired transient buildup if the database is accumulating it.
- [ ] Remember that transients may live in the database or object cache depending on the environment.

Example pattern:

```
$cache_key = 'myplugin_expensive_result_' . md5( $context_id );
$result = get_transient( $cache_key );

if ( false === $result ) {
    $result = myplugin_build_expensive_result( $context_id );
    set_transient( $cache_key, $result, HOUR_IN_SECONDS );
}
```

---

10. REST API, admin-ajax, cookies, and headers

Background requests can reintroduce server load after the initial page is cached.

- [ ] Inspect frontend network requests after page load.
- [ ] Identify calls to:
  - [ ] `/wp-json/...`
  - [ ] `/wp-admin/admin-ajax.php`
  - [ ] cart fragments
  - [ ] analytics/marketing scripts
  - [ ] personalization endpoints
- [ ] Remove polling that is not required.
- [ ] Increase polling intervals where possible.
- [ ] Cache REST responses when safe.
- [ ] Use nonces and authentication correctly for private endpoints.

- [ ] Avoid using `admin-ajax.php` for public high-traffic frontend features when REST endpoints or static rendering would be better.
- [ ] Review headers that prevent caching:
  - [ ] `Cache-Control: no-store`
  - [ ] `Set-Cookie`
  - [ ] `Vary: Cookie`

---

11. Images

Images are often the largest contributor to LCP and page weight.

- [ ] Identify the LCP image on each key template.
- [ ] Ensure LCP image is not lazy-loaded.
- [ ] Serve correctly sized images using `srcset` and `sizes`.
- [ ] Compress images appropriately.
- [ ] Use modern formats where supported: WebP or AVIF.
- [ ] Set explicit width/height to avoid layout shift.
- [ ] Lazy-load below-the-fold images.
- [ ] Avoid huge background images for critical hero content unless intentionally optimized.
- [ ] Preload only the true critical LCP image when needed.

Example:

```
<link rel="preload" as="image" href="/path/to/hero.webp" imagesrcset="/hero-800.webp 800w, /hero-1600.webp 1600w" imagesizes="100vw">
```

Checklist for WordPress themes:

- [ ] Use WordPress image functions where possible so responsive attributes are generated.
- [ ] Avoid hardcoded full-size images.
- [ ] Confirm featured image sizes match actual display requirements.
- [ ] Audit sliders/carousels; they often load multiple large images before interaction.

---

12. CSS, JavaScript, and rendering

Frontend optimization is about reducing render-blocking work and unnecessary main-thread execution.

CSS:

- [ ] Remove unused CSS where practical.
- [ ] Avoid loading site-wide CSS for components used on one page.
- [ ] Inline only small critical CSS when it improves LCP and does not create maintenance risk.
- [ ] Avoid excessive CSS frameworks or duplicate design systems.

JavaScript:

- [ ] Remove unused scripts.
- [ ] Defer non-critical scripts.
- [ ] Delay third-party scripts until interaction when appropriate.
- [ ] Avoid long main-thread tasks.
- [ ] Split heavy functionality by page/template.
- [ ] Prevent duplicate libraries from loading.

WordPress asset checklist:

- [ ] Confirm plugins enqueue assets only where needed.
- [ ] Dequeue unnecessary assets carefully and test affected pages.
- [ ] Avoid breaking dependencies by blindly combining/minifying everything.
- [ ] Check for duplicate jQuery, slider, icon, analytics, or builder assets.

---

13. Fonts and third parties

Fonts and third-party scripts often block rendering or delay interactivity.

Fonts:

- [ ] Use fewer font families, weights, and styles.
- [ ] Prefer system fonts where acceptable.
- [ ] Self-host fonts when it simplifies delivery and caching.
- [ ] Use `font-display: swap` or another intentional strategy.
- [ ] Preload only critical fonts actually used above the fold.
- [ ] Ensure preloaded fonts use matching `type` and `crossorigin` attributes.

Example:

```
<link rel="preload" href="/fonts/inter-var.woff2" as="font" type="font/woff2" cros
```

Third-party scripts:

- [ ] Inventory analytics, ads, chat, maps, embeds, A/B testing, and social widgets.
- [ ] Remove tools that are no longer used.
- [ ] Load third parties after consent or interaction where appropriate.
- [ ] Avoid third-party scripts on templates that do not need them.
- [ ] Measure INP impact after enabling/disabling each major third party.

---

14. Plugins, themes, and page builders

Plugin count alone is not the metric; plugin behavior is.

- [ ] Inventory plugins by function and business value.
- [ ] Identify overlapping plugins.
- [ ] Identify plugins that load assets globally.
- [ ] Identify plugins that run expensive queries or remote calls.
- [ ] Replace heavy plugins with lighter alternatives when the feature is simple.
- [ ] Disable unused modules inside large plugins.
- [ ] Test performance before and after each plugin change.

Page builders and visual editors:

- [ ] Check DOM size.
- [ ] Check nested wrapper depth.
- [ ] Check duplicate CSS/JS libraries.
- [ ] Avoid builder widgets for simple static content when native blocks or theme templates are enough.
- [ ] Rebuild high-traffic landing pages with leaner markup where ROI is clear.

Plugin reduction plan:

1. [ ] List every plugin and its purpose.
2. [ ] Mark must-have, replaceable, unused, and unknown.
3. [ ] Disable one candidate on staging.
4. [ ] Test functionality.
5. [ ] Measure performance delta.
6. [ ] Remove only after confirming no regressions.

---

15. WordPress core configuration

Default WordPress behavior is safe and flexible, but not always optimal for every production site.

- [ ] Disable or limit post revisions only if editorial requirements allow it.
- [ ] Configure autosaves and heartbeat carefully; do not break editing workflows.
- [ ] Disable XML-RPC if not needed.
- [ ] Move WP-Cron to real server cron on production/high-traffic sites.
- [ ] Confirm debug settings are production-safe.
- [ ] Keep core, plugins, and themes updated after testing.

Production `wp-config.php` review:

```
define( 'WP_DEBUG', false );
define( 'SCRIPT_DEBUG', false );
define( 'WP_POST_REVISIONS', 10 );
// Add only after an external cron runner is installed and verified.
define( 'DISABLE_WP_CRON', true );
```

Use with judgment. Do not paste configuration blindly; align it with editorial, hosting, and maintenance needs.

---

16. Cron and background jobs

Scheduled work should not surprise frontend visitors.

- [ ] Identify due and overdue cron events.
- [ ] Move WP-Cron to server cron where appropriate.
- [ ] Avoid heavy cron tasks during peak traffic.
- [ ] Deduplicate scheduled events.
- [ ] Check failed Action Scheduler jobs.
- [ ] For WooCommerce, monitor Action Scheduler queue length and failure rate.
- [ ] Batch long-running jobs.
- [ ] Add locking to prevent overlapping expensive jobs.

Sequence matters. Install and verify the external runner before setting `DISABLE_WP_CRON`. If the constant is set first, scheduled publishing, cleanup jobs, plugin queues, and WooCommerce Action Scheduler jobs can stop until the runner works.

Server cron example:

```
*/5 * * * * cd /path/to/site && wp cron event run --due-now --quiet
```

Verify before changing `wp-config.php`:

```
wp cron event list --due-now
# Check cron logs, PHP logs, or host scheduler logs for successful runs.
```

Only after verification:

```
define( 'DISABLE_WP_CRON', true );
```

After setting the constant, verify due events continue to clear. Roll back by commenting out the constant and clearing OPcache if the backlog grows.

17. WooCommerce and dynamic sites

WooCommerce, membership, LMS, and community sites need careful cache boundaries.

- [ ] Cache catalog and marketing pages aggressively when logged out.
- [ ] Bypass cart, checkout, account, and personalized pages.
- [ ] Check cart fragments or AJAX cart behavior.
- [ ] Reduce unnecessary session creation for anonymous visitors.
- [ ] Optimize product queries and filters.
- [ ] Monitor Action Scheduler.
- [ ] Avoid expensive stock, pricing, or personalization calculations on every page view.
- [ ] Test with real logged-in and logged-out flows after cache changes.

WooCommerce-specific checks:

- [ ] product archive TTFB
- [ ] single product LCP image
- [ ] cart fragments
- [ ] checkout AJAX calls
- [ ] Action Scheduler backlog
- [ ] slow order/admin screens

---

18. Multisite

Multisite performance problems often come from network-wide assumptions.

- [ ] Identify whether slowness is site-specific or network-wide.
- [ ] Audit network-activated plugins.
- [ ] Check global tables and large options.
- [ ] Avoid expensive network-wide queries during normal requests.
- [ ] Review object cache groups and cache segmentation.
- [ ] Confirm domain mapping and redirects are clean.
- [ ] Measure representative subsites separately.

---

19. Monitoring and reporting

Optimization is incomplete without ongoing monitoring.

- [ ] Monitor uptime and response time.
- [ ] Monitor Core Web Vitals over time.
- [ ] Monitor PHP errors and fatals.
- [ ] Monitor database slow queries where supported.
- [ ] Monitor object cache health.
- [ ] Monitor cron/action queues.
- [ ] Monitor CDN cache hit ratio.
- [ ] Keep a change log of performance-related changes.

Client/stakeholder reporting should explain experience, not just scores:

- [ ] What was slow?
- [ ] What work was reduced or moved?
- [ ] What changed in measured results?
- [ ] What risks remain?
- [ ] What should be monitored next?

Example report table:

Area	Before	After	Impact	Next action
Homepage TTFB	900ms	180ms	HTML cache now hitting	Watch cache hit ratio

Area	Before	After	Impact	Next action
Product LCP	4.2s	2.5s	Hero image re-sized/preloac	Audit remaining images
Admin order screen	6.8s	3.1s	Slow query removed	Monitor during peak

---

20. Verification checklist

After each change, retest the same target under the same conditions.

- [ ] Re-run baseline measurements.
- [ ] Compare before/after TTFB.
- [ ] Compare before/after Core Web Vitals.
- [ ] Confirm cache status changed as expected.
- [ ] Confirm no private content is cached publicly.
- [ ] Confirm logged-in workflows still work.
- [ ] Confirm checkout/account/admin flows still work.
- [ ] Check PHP error logs.
- [ ] Check browser console for asset errors.
- [ ] Check visual layout stability.
- [ ] Document the change and result.

If there is no improvement:

- [ ] Confirm you tested the same URL.
- [ ] Confirm caches were warmed or cleared intentionally.
- [ ] Confirm the change reached production/staging.
- [ ] Confirm the bottleneck you fixed was actually dominant.
- [ ] Re-profile instead of stacking more optimizations.

---

21. Recommended optimization order

Use this order to avoid premature or risky changes.

1. [ ] Measure and profile.
2. [ ] Fix redirects, DNS, TLS, and obvious transport issues.
3. [ ] Make full-page cache work for safe public pages.
4. [ ] Remove cache fragmentation from query strings and cookies.
5. [ ] Optimize hosting/runtime bottlenecks.
6. [ ] Fix database, autoload, object cache, and remote HTTP issues.
7. [ ] Optimize images, fonts, CSS, and JavaScript.
8. [ ] Reduce plugin/theme/page-builder overhead.
9. [ ] Move cron/background work out of user requests.
10. [ ] Add monitoring and reporting.

---

22. Quick triage decision tree

High TTFB on logged-out pages

- [ ] Is full-page cache enabled and hitting?
- [ ] Are cookies or query strings bypassing cache?
- [ ] Is the CDN reaching origin unnecessarily?
- [ ] Is WordPress executing for cacheable HTML?

High TTFB on logged-in/admin pages

- [ ] Profile hooks and queries.
- [ ] Check object cache.
- [ ] Check autoloaded options.
- [ ] Check remote HTTP calls.
- [ ] Check plugin-specific admin overhead.

Poor LCP

- [ ] Identify the LCP element.
- [ ] Optimize/preload the LCP image if appropriate.
- [ ] Remove render-blocking CSS/JS.
- [ ] Improve TTFB if the document arrives late.

Poor INP

- [ ] Identify long JavaScript tasks.

- [ ] Reduce third-party scripts.
- [ ] Remove unnecessary frontend plugin assets.
- [ ] Delay non-critical scripts.

Poor CLS

- [ ] Add image/video dimensions.
- [ ] Reserve space for ads/embeds/notices.
- [ ] Avoid late-injected banners above content.
- [ ] Stabilize font loading.

---

23. Definition of done

A performance optimization pass is done when:

- [ ] Key URLs have documented before/after measurements.
- [ ] Improvements are tied to specific causes, not vague plugin changes.
- [ ] Cache behavior is intentional and safe.
- [ ] No critical user flows regressed.
- [ ] Monitoring is in place for the optimized areas.
- [ ] Remaining risks and next steps are documented.

Appendix A: Modern additions (verified against WordPress 6.9.4, 2026-05-18)

The Make WordPress Fast course was recorded before several material WordPress Core performance changes shipped. This appendix covers the most important ones an operator should fold into the checklist above. The unified developer reference at `DEVELOPER_REFERENCE.md` covers each topic in more depth.

Options API: new autoload functions (WordPress 6.4)

WordPress 6.4 introduced first-class functions for managing the autoload state of options. Hand-editing the `autoload` column was always risky; these functions invalidate the right caches and respect the API:

```
wp_set_option_autoload( 'my_huge_option', false );        // singular
wp_set_options_autoload( array( 'opt_a', 'opt_b' ), false ); // bulk
```

Operator workflow:

- [ ] When you find a fat option in §8, flip it via `wp_set_option_autoload()` (or `wp option set-autoload` in WP-CLI), not via raw SQL.
- [ ] Confirm the change with the autoload-size query in §8 — total `autoload_bytes` should drop.

Autoload vocabulary expansion (WordPress 6.6)

As of WordPress 6.6 the `autoload` column holds one of five values: `'on'`, `'off'`, `'auto'`, `'auto-on'`, `'auto-off'`. Old `'yes'`/`'no'` rows persist on upgraded sites and are treated as `'on'`/`'off'`. WordPress 6.6 also added automatic skip-autoload for options larger than ~150 KB by default.

Operator workflow:

- [ ] Replace any saved query, dashboard, or monitor that matches only the legacy `'yes'` autoload value with `WHERE autoload IN ('yes', 'on', 'auto', 'auto-on')`.
- [ ] Check Site Health -> Performance for the "Autoloaded options could affect performance" warning. It triggers at ~800 KB total autoload size.
- [ ] On sites you suspect have stale options, prefer the API/CLI fix over raw deletes; orphaned-option cleanup is a separate maintenance task and should still be done with backups.

Speculation Rules / Speculative Loading (WordPress 6.8)

WordPress 6.8 (April 2025) merged speculative-loading support into Core via the Speculation Rules API. Core's effective default is `prefetch` with `conservative` eagerness, enabled on frontend requests only when pretty permalinks are on and the user is logged out. Core does not ship a dedicated settings UI for speculation rules in 6.8; customization happens through filters such as `wp_speculation_rules_configuration` and `wp_load_speculation_rules`, plus block-level CSS classes like `no-prefetch` and `no-prerender`.

The standalone Speculative Loading plugin extends Core with a plugin-provided admin screen and more aggressive modes, including `prerender`. When you opt into prerender, audit analytics SDKs for prerender-aware behavior and confirm that prerendered visits do not double-count or fire third-party scripts on hidden documents.

Enterprise considerations: edge cache hit ratio can rise on sites with predictable navigation patterns because prefetches may hit cache before the eventual navigation. Confirm session-keyed and personalized pages are excluded from speculative loading with

Core filters, `no-prefetch` / `no-prerender` classes, or plugin-provided exclusion controls. Do not use older `data-prefetch="false"` wording as the recommended Core opt-out mechanism.

Performance Lab plugin

Performance Lab is the Core team's feature-plugin discovery and management layer. As features mature inside the Performance Lab ecosystem they either graduate into Core or remain as standalone plugins; the catalog rotates over time. Verify the current featured plugins on the Performance Lab plugin page before recommending any specific module.

Featured plugins as of 2026-05-18 include:

- Embed Optimizer — improves embeds that otherwise add third-party request and rendering cost.
- Enhanced Responsive Images — refines responsive image sizing decisions, including lazy-loaded images.
- Image Placeholders — formerly Dominant Color Images; uses a CSS background placeholder while an image loads.
- Image Prioritizer — automates `fetchpriority` and related loading decisions for likely LCP candidates.
- Instant Back/Forward — improves browser back/forward-cache behavior where applicable.
- Modern Image Formats — formerly named for WebP uploads; stores additional WebP/AVIF versions of uploaded images and serves modern formats to supporting browsers.
- Optimization Detective — opt-in real-user measurement that informs Core/Performance Lab decisions; dependency for some other featured plugins.
- Performant Translations, Speculative Loading, and View Transitions — additional featured plugins whose relevance depends on the site and rollout risk.

Recommendation: evaluate Performance Lab on staging, look at the current Featured Plugins list at the verification time, and adopt specific plugins by name only when they match the site's documented performance need.

WordPress 6.9 performance deltas

WordPress 6.9 shipped in November 2025; WordPress 6.9.4 was the current patch release verified on 2026-05-18. The most material 6.9 performance changes for this guide are:

- Frontend loading: the WordPress 6.9 Frontend Performance Field Guide covers script `fetchpriority`, footer script-module printing, emoji script-module changes,

block stylesheet handling, hidden-block asset omission, template enhancement output buffering, RSS feed caching, video block CLS fixes, and related frontend work.
- WP-Cron spawn timing: Core now spawns WP-Cron at shutdown rather than earlier in the request lifecycle, reducing user-facing latency for request-triggered cron. Sites with a verified external cron runner remain the preferred high-traffic pattern.
- Query/cache changes: the WordPress 6.9 Field Guide should be checked before relying on plugin code that assumes older query-cache key behavior or intercepts Core caching internals.

Treat this as a delta inventory, not a substitute for measurement: verify impact on the specific site with the same baseline and rollback discipline used elsewhere in this guide.

LCP and `fetchpriority` (WordPress 6.3+)

WordPress 6.3 added automatic `fetchpriority="high"` on the LCP image when Core can identify one. Verify in DevTools before adding manual `<link rel="preload">` for the hero image — Core may already have done it for you.

Core Web Vitals thresholds (post-INP, March 2024)

For reference when reading any current performance report:

Metric	Good	Needs improvement	Poor
LCP	<= 2.5 s	2.5–4.0 s	> 4.0 s
INP	<= 200 ms	200–500 ms	> 500 ms
CLS	<= 0.1	0.1–0.25	> 0.25

INP replaced FID as a Core Web Vital on March 12, 2024. Any report still talking about FID is operating on a 2023 definition.

References

- Options API: Disabling autoload for large options (Make WP Core, June 2024)
- New option functions in 6.4 (Make WP Core)
- WordPress 6.6 Performance Improvements
- Speculative Loading in 6.8 (Make WP Core)
- WordPress 6.8 Performance Improvements

- `wp_set_option_autoload()`
- Performance Lab plugin
- web.dev — Interaction to Next Paint